

Dilema Agen Ganda

- **Batas waktu:** 12 menit.
- **Penyimpanan:** 5 GB
- **Lingkungan:** satu GPU (≈ 16 GB VRAM), tanpa internet
- **Ukuran solusi:** `solution.ipynb` ≤ 1 MB
- **Skor baseline:** 0

Di pusat AI nasional di Astana, dua model komputer — Model R (sebuah ResNet-18) dan Model V (sebuah ViT-Tiny) — sedang menganalisis foto. Saat ini, kedua model bekerja dengan sempurna, mencapai akurasi 100% dan sepakat pada setiap gambar. Untuk menguji seberapa berbeda "otak" pintar mereka sesungguhnya, kepala ilmuwan memberi Anda sebuah tantangan: buatlah perubahan piksel yang sangat kecil dan hampir tidak terlihat pada setiap foto sehingga Model R dan Model V sepenuhnya tidak sepakat (*completely disagree*).



1. Tugas

Dua pengklasifikasi gambar terlatih (*pretrained*) melihat gambar yang sama. Pada gambar-gambar yang disediakan dalam tugas ini, kedua pengklasifikasi bekerja dengan akurasi 100%.

- **Model R:** `torchvision.models.resnet18` (sebuah CNN, ResNet18).
- **Model V:** `vit_tiny_patch16_224` dari `timm` (sebuah Transformer, ViT-Tiny).

Tugas Anda adalah membuat sebuah perubahan kecil ("perturbasi") untuk setiap gambar sehingga kedua model tidak sepakat. Untuk setiap gambar, Anda harus membuat **dua perturbasi yang berbeda**:

- **Tipe A:** setelah ditambahkan, Model R masih mengklasifikasikan gambar dengan benar, tetapi Model V mengklasifikasikannya secara salah.
- **Tipe B:** setelah ditambahkan, Model V masih mengklasifikasikan gambar dengan benar, tetapi Model R mengklasifikasikannya secara salah.

Setiap perturbasi harus *cukup kecil* sehingga sulit diperhatikan. Perturbasi yang lebih kecil mendapatkan skor lebih tinggi (lihat Bagian 5). Perturbasi diterapkan pada gambar asli secara langsung pada tingkat piksel.

2. Data publik

Sekumpulan gambar disediakan bersama tugas ini, tersusun dalam dua split — `train` (100 gambar) dan `test_public` (100 gambar) — masing-masing berisi gambar dengan resolusi yang bervariasi. Semua gambar berasal dari 1000 kelas ImageNet-1K dan baik Model R maupun Model V mencapai akurasi 100% pada kedua split.

Berkas-berkas berikut disediakan:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

Pada saat penilaian, folder `dataset/test_public/` secara transparan digantikan oleh dua himpunan gambar tersembunyi (`test_leaderboard_a` dan `test_leaderboard_b`) untuk penilaian resmi. Masing-masing berisi **100 gambar** dalam format PNG dan sebuah berkas label.

Catatan: Untuk tugas ini, label pada dataset uji dapat diakses.

3. Format keluaran

Untuk setiap gambar, Anda harus menghasilkan dua berkas:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- `{index}` (0, 1, 2, ...), sesuai dengan nama gambar pada dataset.
- Setiap berkas adalah satu tensor tunggal yang disimpan dengan `torch.save`. Bentuknya harus $3 \times H \times W$, di mana `H` dan `W` sesuai dengan **resolusi asli gambar tersebut** (bukan

224 x 224).

- Kode harus menghasilkan hanya satu berkas ZIP, `submission.zip`. Tempatkan semua berkas `.pt` pada tingkat teratas arsip ZIP, tanpa folder pembungkus atau subdirektori.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Notebook akan memberi peringatan jika ada masalah dengan format keluaran.

4. Batasan

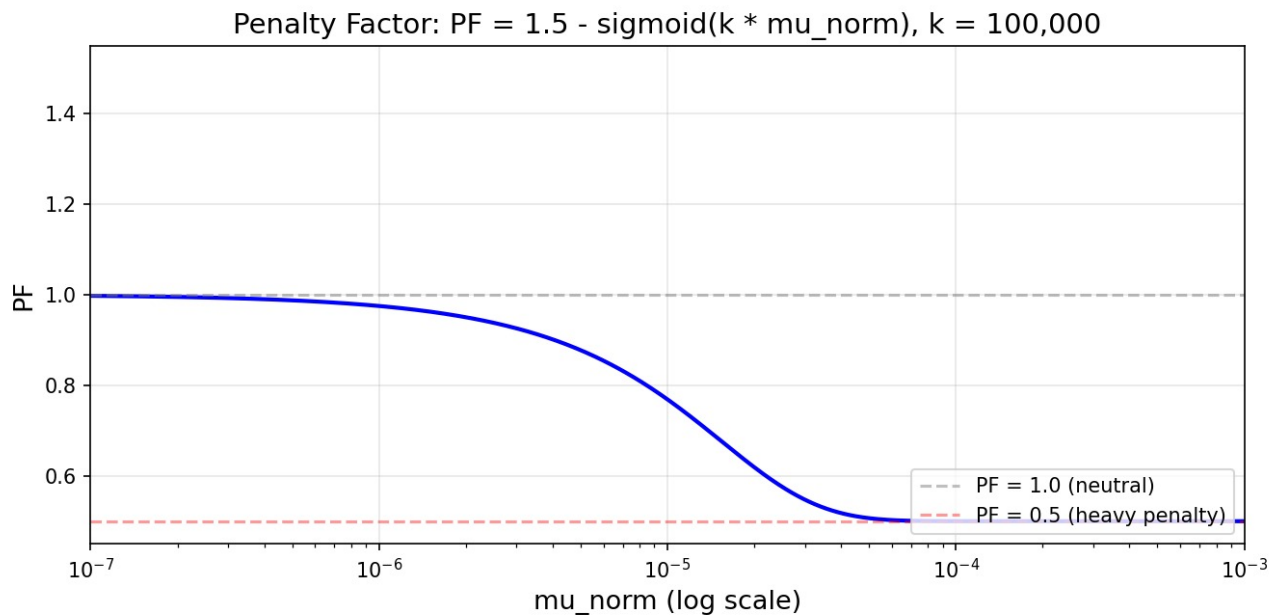
- **Model:** Anda harus menggunakan `torchvision.models.resnet18(pretrained=True)` dan `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Tidak ada model pretrained lain yang diizinkan.
- **Pipeline transformasi (diberlakukan saat evaluasi):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` untuk detailnya.
- **Resolusi perturbasi:** Harus sesuai dengan resolusi gambar mentah **asli** (bukan 224×224). Tensor tersebut ditambahkan ke gambar mentah *sebelum* pipeline transformasi.
- **Format keluaran:** hanya berkas `.pt` — bukan PNG/JPG. Tensor ditambahkan ke gambar mentah dan nilai piksel dipotong (clip) ke `[0, 1]` sebelum prapemrosesan.
- **Penamaan berkas:** Terdaftar secara datar (flat), format `{index}_a.pt / {index}_b.pt` yang ketat. Tidak ada subdirektori di dalam zip.
- **Pustaka:** `torch`, `torchvision`, `timm`.

5. Penilaian

Skor akhir dihitung sebagai berikut. Misalkan M adalah jumlah gambar dalam split, $Score_A$ jumlah perturbasi Tipe A yang berhasil, dan $Score_B$ jumlah perturbasi Tipe B yang berhasil:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF adalah fungsi yang dirancang untuk memberi penalti pada perturbasi dengan norma tinggi dan sangat sensitif di dekat batas atas kinerja. Fungsi ini terbatas pada rentang 0.5 sampai 1. Implementasi lengkapnya dapat dilihat di Bagian 8 pada `solution.ipynb`.



Gambar: Kurva dari fungsi penalti.

6. Memeriksa Pengumpulan

Terdapat pemeriksaan di dalam notebook yang memberi peringatan jika ada masalah format, pada Bagian 7 di notebook `solution.ipynb`.

7. Pengujian lokal

Dokumen `solution.ipynb` berisi contoh lengkap yang berfungsi dengan baik. Berkas ini memuat data publik, kedua model, dan penilai (*scorer*) resmi, serta menulis berkas ZIP pengumpulan. Bacalah sebelum Anda mulai.

8. Cara mengumpulkan

- Simpan perubahan Anda ke `solution.ipynb`.
- Buka tab Git di sidebar kiri JupyterLab.
- **Stage** `solution.ipynb` (ikon + di sebelahnya).
- Masukkan pesan commit dan klik **Commit**.
- Klik ikon awan dengan tanda panah ke atas untuk melakukan push.
- Kembali ke halaman Contest ini dan klik **Submit**.

Kumpulkan tepat satu berkas, bernama `solution.ipynb`.